

EXAMEN LA DISCIPLINA "PROGRAMAREA ALGORITMILOR" – SESIUNEA IANUARIE 2026 –

RESTRICȚII ȘI PRECIZĂRI:

1. Toate subiectele se vor rezolva folosind limbajul Python.
2. Rezolvările vor fi implementate exclusiv folosind facilitățile de bază ale limbajului Python, fără a utiliza module externe.
3. Subiectul 1 este obligatoriu, iar dintre subiectele 2, 3 și 4 se vor rezolva CEL MULT DOUĂ, la alegere.
4. Citirea datelor de intrare se va realiza de la tastatură, iar rezultatele vor fi afișate pe ecran.
5. Se garantează faptul că datele de intrare sunt corecte.
6. Operațiile de sortare se vor efectua folosind funcții sau metode predefinite din limbajul Python.
7. Rezolvările subiectelor alese dintre subiectele 2, 3 și 4 trebuie să conțină:
 - scurtă descriere a algoritmului și o argumentare a faptului că acesta se încadrează într-o anumită tehnică de programare;
 - în cazul problemelor rezolvate folosind metoda Greedy sau metoda programării dinamice se va argumenta corectitudinea criteriului de selecție sau a relațiilor de calcul;
 - în cazul problemelor rezolvate folosind metoda Backtracking se va argumenta relevanța condițiilor de continuare;
 - în cazul subiectelor unde se precizează complexitatea maximă pe care trebuie să o aibă soluția, se va argumenta complexitatea soluției propuse și vor primi punctaj maxim doar soluțiile corecte care se încadrează în complexitatea cerută;
 - în fiecare program Python se va preciza, pe scurt, sub forma unor comentarii, semnificația variabilelor utilizate.
8. Pentru subiectul 1 nu contează complexitatea soluțiilor propuse.
9. Rezolvările corecte care nu respectă restricțiile indicate vor primi punctaje parțiale.
10. Se acordă 1 punct din oficiu.

Subiectul 1 (3 p.)

a) Scrieți o funcție **vocale_distincte** care primește ca parametru un număr variabil de cuvinte (scrise doar cu litere mici din alfabetul englez) și returnează o listă formată din toate cuvintele distincte, sortate alfabetic (lexicografic), dintre cele primite ca parametri, cu proprietatea că au un număr maxim de vocale distincte. De exemplu, pentru apelul **vocale_distincte('aorte', 'steaua', 'aorte', 'areale', 'steaua', 'rea', 'tastatura')** funcția trebuie să furnizeze lista **['aorte', 'steaua']** deoarece aceste cuvinte au un număr maxim de vocale distincte, respectiv 3. **(1.5 p.)**

b) Știind că **puc** este o listă de numere naturale, înlocuiți punctele de suspensie din instrucțiunea **puc = [...]** cu o secvență de inițializare (*list comprehension*) astfel încât, după executarea sa, lista **puc** să conțină toate numerele din lista de numere naturale **numere** care au prima cifră egală cu ultima. De exemplu, pentru **numere = [678, 777, 2101, 9, 2026, 2112, 34543]**, lista **puc** va fi **[777, 9, 2112, 34543]**. **(0.5 p.)**

c) Considerăm următoarea funcție recursivă:

```
def f(s):  
    if len(s) <= 1:  
        return s  
    else:  
        m = len(s) // 2  
        aux = s[:m].replace(s[0], "")  
        return aux + f(s[m+1:])
```

Determinați complexitatea maximă (în cel mai defavorabil caz) a funcției apelată pentru un șir de caractere **s** de lungime **n** astfel: **f(s)**. **(1 p.)**

Subiectul 2 – metoda Greedy (3 p.)

Complexitatea maximă a soluției: $O(n \cdot \log(n))$

Mr. Teve s-a hotărât ca între orele s și t să se uite la TV (orele pot fi numere naturale formate din cel mult 9 cifre, deoarece în lumea lui Mr. Teve ziua poate fi foarte lungă). Mr. Teve și-a făcut o listă cu emisiunile la care s-ar putea uita și intervalele lor de desfășurare. Mr. Teve ar vrea să nu aibă deloc momente libere în care să nu se uite la TV între orele s și t , însă nu ține însă neapărat să vadă o emisiune întreagă, ci poate vedea doar o parte din ea. Totuși, el ar vrea ca numărul de emisiuni la care se uită (parțial sau integral) să fie cât mai mic. Scrieți un program Python care citește de la tastatură s și t (date pe o linie separate prin spațiu), apoi titlul fiecărei emisiunii și intervalul de desfășurare (date ca în exemplu, câte o emisiune pe linie) și afișează un program pentru Mr. Teve cu număr minim de emisiuni, care să nu lase momente libere în intervalul $[s, t]$, sub forma indicată în exemplu (titlul emisiunii și intervalul în care Mr. Teve se va uita la ea). Titlul unei emisiuni nu conține spații și toate orele sunt numere naturale. Dacă nu este posibil ceea ce își dorește Mr. Teve, atunci se va afișa mesajul „Nu se poate!”.

Exemplu:

Date de intrare	Date de ieșire
10 20 Stiri 8 12 Film 10 18 Documentar 17 22 Desene 10 14 Sport 14 20	Film 10 18 Documentar 18 20

Observație: Soluția nu este neapărat unică!

Subiectul 3 – metoda Programării Dinamice (3 p.)

Complexitatea maximă a soluției: $O(n)$

Trenel este pasionat de călătorii. El plănuiește să călătorească în n zile dintr-un an folosind trenul (numărul de zile n dintr-un an din lumea lui Trenel poate avea și 9 cifre). Pentru a face acest lucru, Trenel poate cumpăra **abonamente de călătorie**, fiecare având o anumită durată și un anumit preț. Astfel, există **3 tipuri de abonamente** disponibile, respectiv de 1 zi, 7 zile și 30 de zile, pentru care Trenel știe prețurile. Un abonament de L zile cumpărat în ziua d este valid în ziua d și următoarele $L - 1$ zile. Trenel vrea să cheltuiască **cât mai puțini bani** pentru abonamente, dar nu vrea să călătorească nicio zi fără abonament valid, de aceea vă roagă pe voi să scrieți un program Python care să citească cele 3 prețuri pentru abonamentele de 1, 7, respectiv 30 de zile (prețurile sunt date pe o linie, separate prin spații) și cele n zile în care vrea să călătorească (date pe următoare linie, separate prin spații și în ordine crescătoare), după care să afișeze prețul minim total pe care trebuie să îl plătească Trenel pentru a acoperi **toate zilele** în care călătorește, precum și zilele în care trebuie să își cumpere abonament și ce tip de abonament își cumpără în acea zi, ca în exemplul de mai jos.

Exemplu:

Date de intrare	Date de ieșire
2 7 15 1 4 6 7 8 20	11 Ziua 1 abonament 1 zile Ziua 4 abonament 7 zile Ziua 20 abonament 1 zile

Explicații: Deoarece în zilele 2 și 3 Trenel nu călătorește, el nu are nevoie de un abonament valid. Abonamentul de 7 zile pe care îl cumpără în ziua 4 este valabil în zilele 4, 5, 6, 7, 8, 9, 10.

Subiectul 4 – metoda Backtracking (3 p.)

a) Moș Crăciun are nevoie de m mașinuțe și apelează din nou la cei n spiriduși ai săi. El îl roagă pe fiecare spiriduș i ($1 \leq i \leq n$) să-i spună care este numărul minim a_i și numărul maxim b_i de mașinuțe pe care ar vrea să le asambleze ($a_i \leq b_i$). Moș Crăciun ar vrea să îi pună pe spiriduși să asambleze cele m mașinuțe care-i trebuie pentru Crăciun, dar respectând opțiunile fiecărui spiriduș. Dacă vreți să primiți și voi una dintre mașinuțe, trebuie să-l ajutați pe Moș Crăciun scriind un program Python care să citească de la tastatură numerele m, n și opțiunile a_i și b_i ale fiecăruia dintre cei n spiriduși, după care să afișeze pe ecran toate modalitățile în care Moș Crăciun poate planifica producția de mașinuțe astfel încât spiridușul i să producă un număr de mașinuțe cuprins între a_i și b_i sau mesajul "Imposibil" dacă nu există nicio modalitate de planificare care să respecte cerințele precizate anterior. (2.5 p.)

Exemplu:

Pentru $m = 16, n = 3, a_1 = 1, b_1 = 6, a_2 = 0, b_2 = 7, a_3 = 4, b_3 = 8$ trebuie să fie afișate următoarele 20 de modalități de planificare (nu neapărat în această ordine):

```

1 7 8
2 6 8
2 7 7
3 5 8
3 6 7
3 7 6
4 4 8
4 5 7
4 6 6
4 7 5
5 3 8
5 4 7
5 5 6
5 6 5
5 7 4
6 2 8
6 3 7
6 4 6
6 5 5
6 6 4

```

b) Modificați o singură instrucțiune din program astfel încât să fie afișate doar planificările în care primul și ultimul spiriduș produc același număr de mașini (pentru exemplul anterior, aceste soluții sunt cele subliniate). (0.5 p.)